STOCK PRICE MOVEMENT PREDICTION USING LSTM

Submitted by:

Gowri Shaju

Role Applied:

AI/ML Engineer - L1

Date:

April 4, 2026

Project Overview

This project focuses on predicting stock price movements using historical market data and deep learning techniques. The objective is to model temporal dependencies in stock prices and forecast both the direction and magnitude of future price changes using an LSTM-based architecture.

Contents

1 Introduction **4**

2 Dataset Description **5**
2.1 Data Source 5
2.2 Dataset Features 5
2.3 Nature of the Data 5
2.4 Data Collection and Time Horizon 6
2.5 Data Quality Considerations 6
2.6 Motivation for Feature Selection 6

3 Methodology **7**
3.1 Data Preprocessing 7
3.2 Feature Engineering 7
3.2.1 Technical Indicators 7
3.3 Feature Selection 8
3.4 Sequence Generation 8
3.4.1 Sliding Window Technique 8
3.5 Problem Formulation 8
3.5.1 Classification Task 8
3.5.2 Regression Task 9
3.6 Model Architecture 9
3.6.1 LSTM Network 9
3.6.2 Model Structure 9
3.6.3 Output Layers 9

4 Model Training **10**
4.1 Data Splitting Strategy 10
4.2 Batch Processing 10
4.3 Loss Functions 10
4.3.1 Classification Loss 10
4.3.2 Regression Loss 11
4.3.3 Combined Loss 11
4.4 Optimization Algorithm 11
4.5 Training Procedure 11
4.6 Regularization Techniques 11
4.6.1 Dropout 12
4.6.2 Early Stopping 12
4.7 Reproducibility 12

5 Hyperparameter Tuning **13**
5.1 Importance of Hyperparameter Tuning 13
5.2 Hyperparameters Considered 13
5.3 Tuning Strategy 14
5.4 Validation-Based Selection 14
5.5 Final Hyperparameter Configuration 14
5.6 Challenges in Tuning 14

6 Evaluation Metrics **15**
6.1 Classification Metrics . . . 15
6.1.1 Accuracy . . . 15
6.1.2 Precision . . . 15
6.1.3 Recall . . . 15
6.1.4 F1-Score . . . 15
6.1.5 ROC-AUC Score . . . 16
6.1.6 Confusion Matrix . . . 16
6.2 Regression Metrics . . . 16
6.2.1 Mean Absolute Error (MAE) . . . 16
6.2.2 Mean Squared Error (MSE) . . . 16
6.2.3 Root Mean Squared Error (RMSE) . . . 16
6.3 Model Evaluation Strategy . . . 17
6.4 Interpretation of Metrics . . . 17

7 Results and Analysis **18**
7.1 Quantitative Results . . . 18
7.1.1 Classification Results . . . 18
7.1.2 Regression Results . . . 18
7.2 Classification Analysis . . . 18
7.3 Visualization of Classification Results . . . 19
7.4 Regression Analysis . . . 20
7.5 Visualization of Regression Results . . . 20
7.6 Discussion . . . 20

8 Limitations **21**

9 Conclusion **22**

1 Introduction

Financial markets produce large volumes of time-dependent data, making them a natural domain for time series analysis and predictive modeling. Predicting stock price movements is a key task in quantitative finance, as it supports decision-making in areas such as trading, portfolio management, and risk assessment. However, stock prices are influenced by complex and dynamic factors including market trends, investor sentiment, and macroeconomic conditions, making accurate prediction inherently challenging.

The objective of this project is to develop a deep learning-based model that leverages historical market data to predict short-term stock price movements. Specifically, the model aims to learn temporal patterns in sequential data and use them to forecast future behavior. To achieve this, Long Short-Term Memory (LSTM) networks are employed, as they are well-suited for capturing long-range dependencies in time series data.

The dataset used in this study is AAPL data obtained from Yahoo Finance and consists of daily stock data, including Open, High, Low, Close, Adjusted Close, and Volume. In addition to raw price data, feature engineering techniques are applied to generate technical indicators that capture trends, momentum, and volatility. The data is then transformed into fixed-length sequences using a sliding window approach to meet the input requirements of the LSTM model.

The problem is formulated as a dual-task learning objective, combining both classification and regression. The model simultaneously predicts:

- The direction of future price movement (upward or downward)
- The magnitude of the price change over a fixed time horizon

This joint formulation provides a more comprehensive understanding of market behavior compared to single-task approaches.

The classification task predicts whether the future return is positive or negative, while the regression task estimates the magnitude of this return. This formulation ensures that the model learns a true forward-looking prediction task without data leakage and aligns closely with real-world financial decision-making scenarios.

The overall system is designed in a modular manner, with clearly defined stages including data preprocessing, feature engineering, sequence generation, model training, and evaluation. This structured approach enhances reproducibility and allows for easy experimentation with alternative features and model architectures.

2 Dataset Description

2.1 Data Source

The dataset used in this project consists of historical stock market data of Apple obtained from Yahoo Finance, a widely used financial data platform that provides reliable and publicly accessible market information. The data represents the daily trading activity of a selected stock (e.g., Reliance Industries), making it suitable for time series forecasting tasks.

The dataset captures chronological information about stock price movements and trading behavior, which is essential for modeling temporal dependencies using deep learning techniques such as Long Short-Term Memory (LSTM) networks.

2.2 Dataset Features

Each record in the dataset corresponds to a single trading day and includes the following key attributes:

- **Open Price**: The price of the stock at the beginning of the trading session.
- **High Price**: The maximum price reached during the trading day.
- **Low Price**: The minimum price recorded during the trading day.
- **Close Price**: The final price of the stock at the end of the trading session.
- **Volume**: The total number of shares traded during the day.

These features provide a comprehensive view of daily market activity and are commonly used in financial time series analysis.

2.3 Nature of the Data

The dataset is inherently a **time series dataset**, where observations are ordered chronologically. Unlike traditional datasets, time series data exhibits temporal dependencies, meaning that past values influence future values.

Key characteristics of the dataset include:

- **Temporal Dependency**: Stock prices depend on historical trends and patterns.
- **Non-linearity**: Market behavior is highly complex and cannot be modeled using simple linear relationships.
- **Noise and Volatility**: Stock prices are affected by external factors such as news, economic conditions, and investor sentiment, leading to unpredictable fluctuations.
- **Non-stationarity**: Statistical properties such as mean and variance may change over time.

These characteristics make stock price prediction a challenging task and justify the use of advanced models like LSTM.

2.4 Data Collection and Time Horizon

The dataset spans multiple years of historical stock data, providing sufficient observations for training and evaluating the model. A longer time horizon allows the model to learn diverse market conditions, including bullish, bearish, and sideways trends. The data is collected at a daily frequency, which is appropriate for short-term prediction tasks and reduces the noise typically present in higher-frequency (intraday) data.

2.5 Data Quality Considerations

Before using the dataset for model training, several data quality aspects are considered:

- **Missing Values**: Any missing or null entries are identified and handled appropriately to ensure consistency.
- **Data Consistency**: Ensuring that all records follow chronological order without duplication.
- **Outliers**: Extreme values are analyzed to determine whether they represent genuine market movements or data errors.

Proper handling of these issues is critical to avoid misleading patterns during model training.

2.6 Motivation for Feature Selection

The selected features (Open, High, Low, Close, Volume) are fundamental indicators of market activity and are widely used in financial analysis. These features capture both price dynamics and trading intensity, which are essential for predicting future movements.

In addition to these raw features, derived indicators such as moving averages and Relative Strength Index (RSI) are incorporated during the feature engineering phase to enhance the model's ability to detect trends and momentum.

3 Methodology

This chapter outlines the entire pipeline utilized in this project, encompassing data preprocessing, feature engineering, sequence generation, problem formulation, and model architecture. The methodology is made to make sure that the model accurately captures time-based relationships in stock price data while still being strong and generalizable.

3.1 Data Preprocessing

Data preprocessing is a critical step in preparing raw financial data for model training. Since stock market data is inherently noisy and non-stationary, appropriate preprocessing techniques are applied to improve data quality and model performance.

Handling Missing Values

The dataset is first examined for missing or null values. Any missing entries are handled using appropriate techniques such as forward filling or removal, depending on the extent of missing data. This ensures continuity in the time series.

Feature Scaling

Stock price features vary significantly in magnitude, which can negatively affect the convergence of deep learning models. Therefore, feature scaling is applied using Min-Max normalization. This transforms all features into a uniform range, typically between 0 and 1, ensuring stable and faster training.

Chronological Ordering

To preserve temporal integrity, the dataset is strictly maintained in chronological order. This prevents any leakage of future information into the training phase, which is crucial for realistic time series modeling.

3.2 Feature Engineering

Feature engineering enhances the predictive power of the model by incorporating additional informative variables derived from raw data.

3.2.1 Technical Indicators

In addition to basic price features, technical indicators are computed to capture market trends and momentum. These include:

- Moving Averages (short-term and long-term)
- Relative Strength Index (RSI)
- Price change (difference between consecutive closing prices)

These indicators provide the model with richer contextual information about market behavior.

Target Engineering: In addition to feature generation, the target variable for classification is carefully designed to reflect meaningful market behavior. A moving average-based approach is used, where the closing price is compared against its 20-day rolling mean. This helps in identifying trend direction rather than short-term fluctuations.

This design choice improves the stability of the classification task and reduces the effect of noise inherent in financial time series data.

3.3 Feature Selection

Relevant features are selected based on their importance in capturing price dynamics. Irrelevant or redundant features are excluded to reduce model complexity and improve generalization.

3.4 Sequence Generation

Since LSTM models require sequential input data, the time series is transformed into overlapping sequences using a sliding window approach.

3.4.1 Sliding Window Technique

A fixed window of past observations is used as input to predict future values. For example, the model may use the previous n days of data to predict the next day's movement.

Formally, if X_t represents the data at time t, the input sequence is defined as:

$$(X_{t-n}, X_{t-n+1}, \ldots, X_{t-1}) \rightarrow X_t$$

This transformation converts the dataset into a supervised learning format suitable for LSTM training.

3.5 Problem Formulation

The prediction task is formulated as both a classification and regression problem.

3.5.1 Classification Task

The classification component predicts the future direction of stock price movement over a fixed prediction horizon.

Let C_t denote the closing price at time t, and let C_{t+h} denote the closing price at a future time step, where h is the prediction horizon. The future return is defined as:

$$r_t = \frac{C_{t+h}}{C_t} - 1$$

The target direction is then defined as:

$$\text{Target Direction} = \begin{cases} 1 & \text{if } r_t > 0 \\ 0 & \text{otherwise} \end{cases}$$

By basing the target on future returns, the model learns to predict actual future movements rather than relying on contemporaneous indicators, making the approach more realistic and aligned with practical trading scenarios. Additionally, this formulation ensures consistency between the classification and regression tasks, as both are derived from the same future return signal.

3.5.2 Regression Task

The regression component predicts the magnitude of price change, defined as the difference between consecutive closing prices. This provides insight into the strength of the predicted movement.

3.6 Model Architecture

The core of the proposed system is a Long Short-Term Memory (LSTM) network, which is well-suited for modeling sequential data.

3.6.1 LSTM Network

LSTM networks are capable of learning long-term dependencies using memory cells and gating mechanisms. These gates regulate the flow of information, allowing the model to retain relevant patterns and discard noise.

3.6.2 Model Structure

The architecture consists of the following components:

- Input layer accepting sequential data
- One or more LSTM layers with configurable hidden units
- Dropout layers for regularization
- Fully connected (Dense) layers

3.6.3 Output Layers

The model produces two outputs:

- A classification output using a sigmoid activation function
- A regression output using a linear activation function

This dual-output design enables the model to simultaneously predict direction and magnitude.

4 Model Training

This chapter describes the training process of the proposed LSTM-based model, including data splitting strategy, loss functions, optimization techniques, and regularization methods. Proper training is essential to ensure that the model learns meaningful patterns from the data while avoiding overfitting.

4.1 Data Splitting Strategy

To evaluate the model in a realistic setting, the dataset is divided into three subsets: training, validation, and testing. Unlike traditional machine learning tasks, random splitting is not suitable for time series data due to temporal dependencies.

Instead, a chronological split is used:

- **Training Set**: Used to train the model on historical data
- **Validation Set**: Used for tuning hyperparameters and monitoring performance
- **Test Set**: Used for final evaluation on unseen data

This approach ensures that future information is not used during training, thereby preventing data leakage and maintaining the integrity of the evaluation process.

4.2 Batch Processing

The training data is divided into batches to improve computational efficiency and stabilize gradient updates. Batch processing allows the model to learn incrementally and is particularly useful when working with large datasets.

The batch size is treated as a tunable hyperparameter and is selected based on performance and available computational resources.

4.3 Loss Functions

Since the model performs both classification and regression tasks, separate loss functions are used for each output.

4.3.1 Classification Loss

For the classification task (predicting price direction), Binary Cross-Entropy (BCE) loss is used:

$$\mathcal{L}_{\text{classification}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

where y_i is the true label and $\hat{y}_i$ is the predicted probability.

4.3.2 Regression Loss

For the regression task (predicting magnitude of change), Mean Squared Error (MSE) loss is used:

$$\mathcal{L}_{\text{regression}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

4.3.3 Combined Loss

The total loss is computed as a weighted combination of classification and regression losses:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{classification}} + \lambda_2 \mathcal{L}_{\text{regression}}$$

where λ_1 and λ_2 are weighting factors that balance the importance of each task.

4.4 Optimization Algorithm

The model is trained using the Adam optimizer, which is an adaptive learning rate optimization algorithm widely used in deep learning. Adam combines the advantages of both momentum and RMSProp, enabling faster convergence and better handling of sparse gradients.

The learning rate is treated as a critical hyperparameter and is tuned to achieve optimal performance.

4.5 Training Procedure

The training process involves multiple epochs, where each epoch represents one complete pass through the training dataset.

For each epoch:

- The model processes all training batches
- Loss is computed and gradients are backpropagated
- Model weights are updated using the optimizer
- Validation loss is monitored to track performance

Training continues until a predefined number of epochs is reached or until convergence criteria are satisfied.

4.6 Regularization Techniques

To improve generalization and prevent overfitting, several regularization techniques are employed:

4.6.1 Dropout

Dropout layers are added to the model to randomly deactivate neurons during training. This prevents the model from relying too heavily on specific features and encourages better generalization.

4.6.2 Early Stopping

Early stopping is used to terminate training when the validation loss stops improving. This helps prevent overfitting and reduces unnecessary computation.

4.7 Reproducibility

To ensure consistent and reproducible results, random seeds are fixed during training. This controls sources of randomness such as weight initialization and data shuffling.

5 Hyperparameter Tuning

Hyperparameter tuning is a crucial step in optimizing the performance of the proposed LSTM-based model. Unlike model parameters, which are learned during training, hyperparameters are predefined and significantly influence the learning process, convergence behavior, and generalization capability of the model.

In this project, a systematic approach is adopted to tune key hyperparameters in order to achieve optimal predictive performance while avoiding overfitting.

5.1 Importance of Hyperparameter Tuning

The performance of deep learning models is highly sensitive to the choice of hyperparameters. Poorly selected hyperparameters can lead to issues such as slow convergence, underfitting, or overfitting. Therefore, careful tuning is necessary to ensure that the model learns meaningful patterns from the data and performs well on unseen samples.

5.2 Hyperparameters Considered

The following hyperparameters are tuned during the experimentation process:

Number of LSTM Layers

The depth of the network determines its ability to learn complex patterns. Increasing the number of layers can improve representation power but may also increase the risk of overfitting and computational cost.

Number of Hidden Units

The number of hidden units in each LSTM layer controls the capacity of the model. A higher number of units allows the model to capture more complex relationships but may lead to overfitting if not properly regularized.

Learning Rate

The learning rate determines the step size during gradient updates. A very high learning rate may cause the model to diverge, while a very low learning rate can result in slow convergence. Selecting an appropriate learning rate is critical for stable training.

Batch Size

Batch size affects the stability and efficiency of training. Smaller batch sizes provide more frequent updates but may introduce noise, while larger batch sizes improve computational efficiency but require more memory.

Dropout Rate

Dropout is used as a regularization technique to prevent overfitting. The dropout rate determines the fraction of neurons that are randomly deactivated during training.

5.3 Tuning Strategy

A combination of manual tuning and experimental analysis is employed to identify optimal hyperparameter values. Different configurations are tested, and their performance is evaluated using the validation dataset.

The tuning process involves:

- Selecting a range of values for each hyperparameter
- Training the model with different combinations
- Monitoring validation loss and evaluation metrics
- Selecting the configuration that provides the best trade-off between accuracy and generalization

5.4 Validation-Based Selection

The validation dataset plays a key role in hyperparameter tuning. Model performance is evaluated on the validation set after each training run, and hyperparameters are adjusted accordingly.

This approach ensures that the model does not overfit to the training data and maintains good generalization performance.

5.5 Final Hyperparameter Configuration

After experimentation, an optimal set of hyperparameters is selected based on validation performance. These parameters are then used to train the final model, which is evaluated on the test dataset.

The selected configuration achieves a balance between model complexity, training efficiency, and predictive accuracy.

5.6 Challenges in Tuning

Hyperparameter tuning in deep learning models presents several challenges:

- High computational cost due to multiple training runs
- Sensitivity of model performance to small changes in parameters
- Trade-off between model complexity and generalization

Despite these challenges, careful tuning significantly improves the overall performance and stability of the model.

6 Evaluation Metrics

To comprehensively evaluate the performance of the proposed model, both classification and regression metrics are employed. Since the model predicts both the direction and magnitude of stock price movement, separate evaluation criteria are used for each task.

6.1 Classification Metrics

The classification component of the model predicts whether the stock price will move upward or downward. The following metrics are used to evaluate classification performance:

6.1.1 Accuracy

Accuracy measures the proportion of correctly predicted instances out of the total number of predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- TP: True Positives
- TN: True Negatives
- FP: False Positives
- FN: False Negatives

6.1.2 Precision

Precision measures the proportion of correctly predicted positive observations among all predicted positives:

$$\text{Precision} = \frac{TP}{TP + FP}$$

6.1.3 Recall

Recall measures the proportion of actual positive cases that are correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN}$$

6.1.4 F1-Score

The F1-score is the harmonic mean of precision and recall:

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

6.1.5 ROC-AUC Score

The Receiver Operating Characteristic - Area Under Curve (ROC-AUC) is a threshold-independent metric used to evaluate the performance of a binary classification model.

The ROC curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at different threshold values:

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

The Area Under the Curve (AUC) represents the probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative instance.

$$\text{AUC} = \int_0^1 \text{TPR}(x)\, dx$$

A higher AUC value indicates better model performance:

- AUC = 1.0 : Perfect classifier
- AUC = 0.5 : Random guessing

ROC-AUC is particularly useful in evaluating model performance when class distribution is imbalanced, as it considers all possible classification thresholds.

6.1.6 Confusion Matrix

A confusion matrix provides a detailed breakdown of prediction results, including true positives, true negatives, false positives, and false negatives. It helps in analyzing model bias and error distribution.

6.2 Regression Metrics

The regression component predicts the magnitude of stock price change. The following metrics are used:

6.2.1 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$$

6.2.2 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

6.2.3 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2}$$

6.3 Model Evaluation Strategy

The model is evaluated on the test dataset, which consists of unseen data not used during training or validation. For classification, predicted probabilities are converted into binary labels using a threshold value (typically 0.5). For regression, predicted values are directly compared with actual values.

6.4 Interpretation of Metrics

A good classification model should achieve high accuracy, precision, recall, and F1-score, indicating balanced and reliable predictions. A good regression model should have low MAE, MSE, and RMSE, indicating minimal prediction error.

Due to the inherent volatility of stock market data, results are interpreted with consideration of real-world uncertainty.

7 Results and Analysis

This section presents the quantitative results obtained from the proposed LSTM-based model, followed by a detailed analysis of its performance on both classification and regression tasks.

7.1 Quantitative Results

The performance of the model is evaluated using standard classification and regression metrics.

7.1.1 Classification Results

Metric	Value
Accuracy	0.60
Precision	0.62
Recall	0.74
F1 Score	0.68
ROC-AUC	0.59

Table 1: Classification Performance Metrics

7.1.2 Regression Results

Metric	Value
MAE	0.06
RMSE	0.07

Table 2: Regression Performance Metrics

7.2 Classification Analysis

The classification results indicate that the model achieves moderate predictive performance. With an accuracy of approximately 60%, the model performs better than random guessing, suggesting the presence of meaningful predictive signals in the data.

The recall is relatively higher than precision, indicating that the model is more effective at identifying upward movements but may produce false positives. This behavior is common in financial datasets where class imbalance or trend persistence exists.

The ROC-AUC score of approximately 0.59 further confirms that while the model captures some structure in the data, its discriminative ability remains limited.

7.3 Visualization of Classification Results

The following plots provide a deeper understanding of the classification performance.

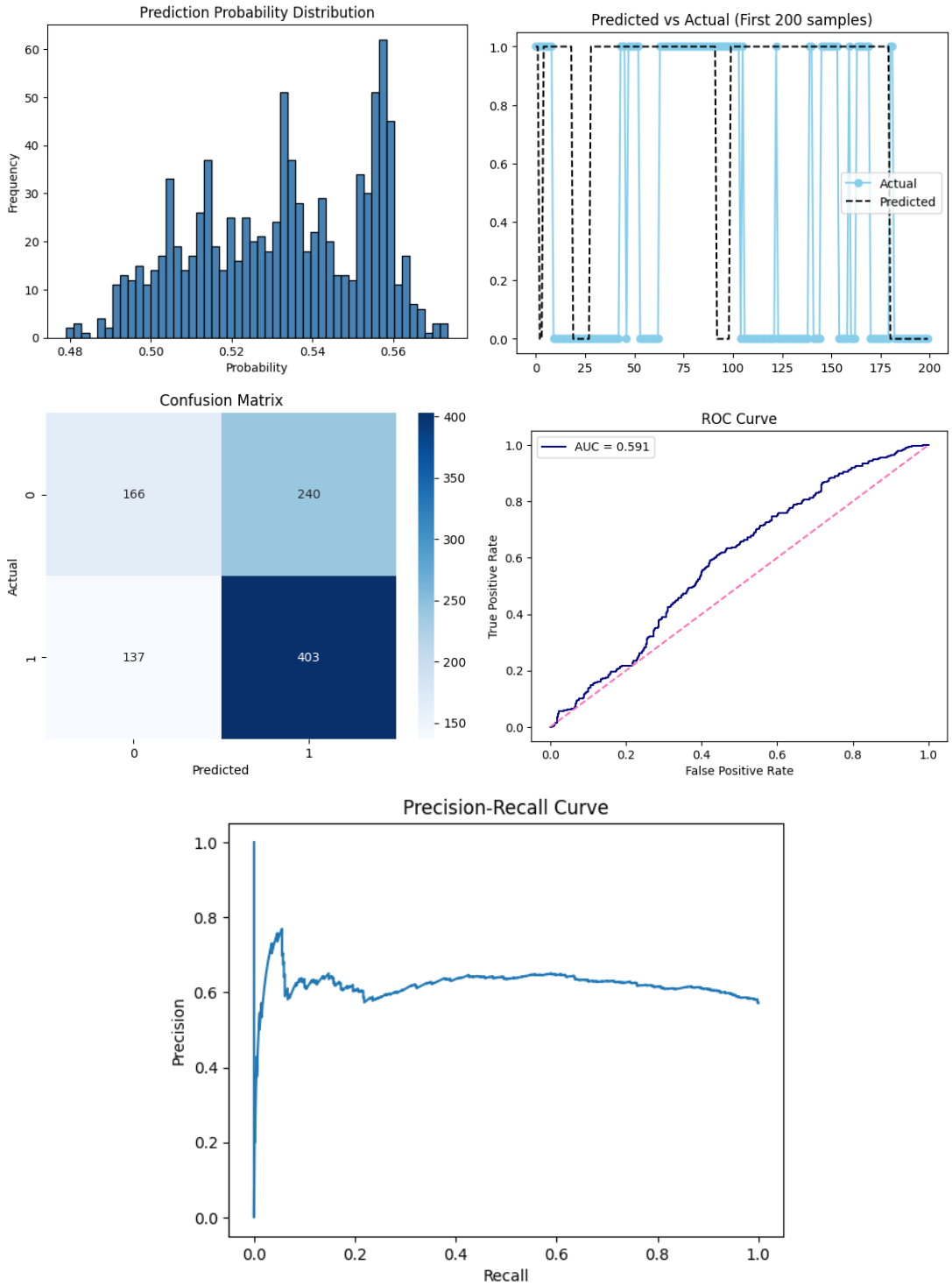

Figure 1: Classification Performance Visualizations

7.4 Regression Analysis

The regression component predicts the magnitude of future returns. The results indicate that:

- The model captures general trends in price movement.
- Prediction errors remain relatively small in stable market conditions.
- The model struggles to accurately predict sharp spikes or sudden drops.

This behavior is expected in financial time series due to high volatility and noise.

7.5 Visualization of Regression Results

The regression performance is analyzed using the following plots.

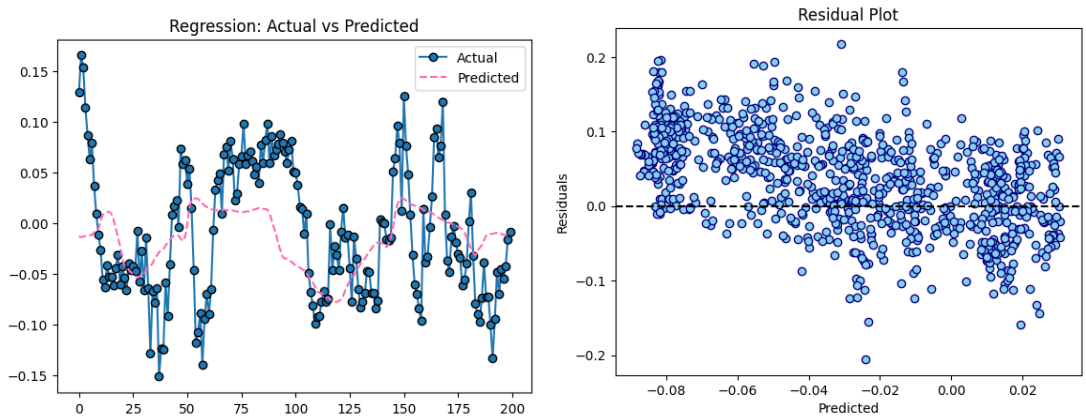

Figure 2: Regression Performance Visualizations

7.6 Discussion

The results highlight that the LSTM model is capable of learning temporal dependencies in stock price data, but its predictive power is inherently limited.

The classification accuracy of around 60% indicates that the model captures useful directional signals, though not strong enough for highly reliable predictions. This level of performance is considered realistic in financial forecasting, where achieving very high accuracy is uncommon.

The regression results further demonstrate that while the model can approximate general movement, it tends to smooth extreme variations, making it less effective during volatile periods.

Overall, the model provides a reasonable baseline for stock movement prediction while maintaining a realistic and leakage-free setup.

8 Limitations

Despite achieving moderate performance, the proposed LSTM-based model has several limitations:

- **Market Noise and Volatility:** Financial data is highly noisy and influenced by external factors such as news, economic events, and investor sentiment, which are not captured in the model.
- **Limited Predictive Power:** The model achieves only moderate accuracy (60%), reflecting the inherent difficulty of stock market prediction.
- **Weak Regression Performance:** Predicting the exact magnitude of price changes remains challenging due to high variability in returns.
- **Smoothing of Extreme Events:** The model tends to smooth sharp price movements, making it less effective in capturing sudden spikes or crashes.
- **Limited Feature Set:** The model relies primarily on historical price-based features and technical indicators, without incorporating external data such as sentiment or macroeconomic variables.
- **Sensitivity to Hyperparameters:** Model performance is highly dependent on hyperparameter choices, requiring careful tuning.
- **Computational Cost:** Training and tuning deep learning models require significant computational resources.

9 Conclusion

In this project, a deep learning-based approach for stock price movement prediction was developed using Long Short-Term Memory (LSTM) networks. The model was designed to jointly predict both the direction and magnitude of future price movements.

The results demonstrate that the model achieves moderate predictive performance, with classification accuracy around 60%. While this may appear modest, it is important to note that financial markets are highly noisy, non-linear, and influenced by unpredictable external factors.

The model successfully captures general trends in stock movement and provides meaningful directional signals, indicating the presence of learnable patterns in historical data. However, its ability to predict precise price changes remains limited, particularly during volatile market conditions.

This project emphasizes the importance of realistic problem formulation and evaluation in financial machine learning. By avoiding data leakage and using future-based targets, the model reflects a practical forecasting scenario.

Overall, the work highlights both the potential and the limitations of deep learning in financial prediction, providing a strong foundation for future improvements through better features, advanced architectures, and incorporation of external data sources.